

Série des exercices N°1

Exercice 1 :

Donner le résultat du programme ci-dessous, quel est son rôle en sciences de données ?

```
import re
def normaliser(s: str) -> str:
    # strip + lower
    s = s.strip().lower()
    # supprimer tous les caractères non alphabétiques
    s = re.sub(r"[^a-z]+", "", s)
    return s

messages = [" Bonjour!!! ", "HELLO ", "sAlUt-----", " Master??? ", " bonjour "]
messages_norm = [normaliser(m) for m in messages]
print(messages_norm)
```

Exercice 2 : Exécuter manuellement le code suivant et donner son résultat et son rôle

```
import re
lignes = [ "ID=103; Nom=Karim Noufal; Age=27; Email=karim.nf@gmail.com",
           "ID=104; Nom=Sara Haddad; Age=31; Email=s.haddad@fsbm.ma",
           "ID=108; Nom=Hicham; Age=?; Email=hicham@invalid" ]
# -----
# 1) Regex pour les emails
# -----
regex_email = re.compile(r"^[w\.-]+@[w\.-]+\.[a-z]{2,}$", re.IGNORECASE)
# -----
def alpha_or_espace(ch):
    for e in ch:
        if not (e.isalpha() or e.isspace()): return False
    return True
# 2) Fonction de parsing d'une ligne
# -----
def parse_line(s: str) -> list:
    resultat = []
    # Étape 1 : découper aux ';'
    parties = s.split(';')
    for partie in parties:      # enlever espaces inutiles
        partie = partie.strip()
```

```

# clé-valeur
if '=' in partie:
    cle, valeur = partie.split('=')
    cle = cle.strip()
    valeur = valeur.strip()
    resultat += [[cle, valeur]]

# Étape 2 : validation Id
id = resultat[0][1]
if not id.isdecimal(): resultat[0][1] = "invalide"

# Étape 3 : validation Nom
Nom = resultat[1][1]
if not (alpha_or_espace(Nom)): resultat[1][1] = "invalide"

# Étape 4 : validation Nom
Age = resultat[2][1]
if not (Age.isdecimal()): resultat[2][1] = "invalide"
else: resultat[2][1] = int(resultat[2][1])

# Étape 5 : validation Email
email = resultat[3][1]
if not bool(regex_email.match(email)): resultat[3][1] = "invalide"
return resultat

# -----
# 3) Application à toutes les lignes
# -----
parsed = [parse_line(ligne) for ligne in lignes]

def valider_infos(X):
    for e in X:
        if e[1] == "invalide":
            X.append("=====> Ligne à supprimer")
        return X
    X.append("=====> Ligne valide")
    return X

parsed_validation = [valider_infos(ligne) for ligne in parsed]
# -----
# 4) Affichage final
# -----
for obj in parsed_validation: print(obj)

```

Exercice 3 :

ANALYSEUR LEXICAL D'UN LANGAGE

Présentation. Avant son exécution, tout code source d'un programme informatique doit être traduit en un autre code appelé code machine. Cette traduction est réalisée par un compilateur dont le rôle est de transformer le code source en une suite d'instructions élémentaires directement exécutables par le processeur.

Ainsi le rôle de l'analyseur lexical est d'identifier puis supprimer les caractères superflus du code source (commentaires, espaces, ..) et de reconnaître les mots clés, les identificateurs, les opérateurs qui sont définis par un ensemble de règles.

En plus, l'analyseur lexical signale les éventuelles erreurs de syntaxe et associe à chaque erreur le numéro de ligne dans laquelle elle intervient.

Dans ce problème, on se propose de mettre en œuvre un analyseur lexical. Il s'agit d'implémenter des fonctions en langage PYTHON pour un analyseur lexical d'un pseudo langage informatique noté PL.

A- Gestion des commentaires et des espaces

Question A-1 : test d'un commentaire

Un commentaire est une instruction qui n'est pas traduite par le compilateur. Pour ce pseudo langage PL, un commentaire est une chaîne de caractères qui doit commencer obligatoirement par les 2 caractères * (antislash étoile) et se terminer par les 2 caractères *\ (étoile antislash).

- ⊙ Ecrire une fonction ***comment(instr)*** qui retourne True si la chaîne de caractères instr est un commentaire du langage LIM et False sinon.

Exemple :

- soit la chaîne de caractères instr= "* explication *\ " alors l'appel de la fonction ***comment(instr)*** retourne True.
- si la chaîne de caractères instr= "explication" alors l'appel ***comment(instr)*** retourne False.

Question A-2- Suppression d'espaces multiples dans une instruction

Dans une instruction, on a tendance à supprimer les espaces multiples qui se succèdent et de ne conserver qu'un seul espace.

Exemple. La chaîne de caractères inst= ' x = 21 ' ne doit pas être traitée par le processeur avant de supprimer les espaces superflus. On doit avoir le résultat suivant : inst= 'x = 21'.

- ⊙ Ecrire une fonction ***suppEspaces(inst)*** qui supprime les espaces multiples consécutifs (qui se suivent) entre les caractères de la chaîne ***inst*** en paramètre. S'il y'a plusieurs espaces au début ou à la fin, on ne conserve aucun.

B : Reconnaissance des mots-clés et des identificateurs.

Tout langage de programmation définit un ensemble de mots clés qui sont des chaînes de caractères ayant des significations et des utilisations spécifiques pour ce langage.

Pour notre pseudo langage PL, on suppose avoir défini une liste **MotsCles** qui contient les chaînes de caractères qui représentent les mots clés du notre pseudo langage PL.

Par exemple : **MotsCles** = ['si', 'sinon', 'tantque', 'pour', 'definir'].

Question B-1 : Vérification d'un mot clé

- ⊙ Ecrire une fonction **motCle(mot, LMotsCles)** qui retourne True si **mot** (le paramètre de la fonction) est un mot clé de PL et False sinon.

Exemple : l'appel **motCle("si")** retourne **True** et l'appel **motcle("entier")** retourne **False**.

Question B-2 : Validité d'un identificateur

Un identificateur est une chaîne de caractères qui permet de définir et d'identifier les éléments d'un programme (les variables, les fonctions, ..). Il doit respecter un ensemble de règles particulières pour chaque langage.

Un identificateur du pseudo langage PL est valide s'il respecte les quatre conditions (C1, C2, C3 et C4) suivantes :

C1- Sa longueur est inférieure strictement à 80

C2- Ne contient aucun espace

C3- Ne commence pas par un chiffre ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9').

C4- Ne doit pas être un mot clé du langage PL (n'est pas dans **MotsCles**)

- ⊙ Ecrire une fonction **identificateur(id)** qui retourne True si son paramètre (la chaîne de caractères **id**) est un identificateur valide de PL et False sinon.

C : Implémentation de l'analyseur lexical.

Question C-1 : Analyse d'une instruction

Une instruction du langage PL est correcte si elle vérifie l'une des conditions suivantes :

- L'instruction est un commentaire du langage PL (voir question A-1).
- L'instruction commence par un identificateur valide suivi d'un espace suivi d'une chaîne de caractères non vide. On doit supprimer les espaces superflus.
- L'instruction commence par un mot clé du langage PL suivi par un espace, suivi d'une chaîne de caractères qui se termine par le caractère ' ; ' (point virgule).

- ⊙ Ecrire une fonction **analyserInstruction(inst)** qui retourne True si la chaîne de caractères **inst** en paramètre correspond à une instruction correcte de PL et False sinon.

Exercice 4: COMPRESSION ET DECOMPRESSION DES CHAINES DE CARACTERES.

Définitions :

- **La compression** de donnée est le traitement informatique utilisant un algorithme particulier, qui permet de transformer une suite de bits A en une suite de bits B plus courte. Les suites A et B contiennent les mêmes informations, mais codées d'une manière différente.
- **La décompression** est l'opération inverse de la compression.
- **L'algorithme RLE (Run-Length Encoding)**, appelé en français **le codage par plages**, est un algorithme de compression informatique qui consiste à repérer et à éliminer la redondance des données. Toute suite de bits ou de caractères identiques est remplacée par un couple (caractère répété suivi de son nombre d'occurrences (son nombre de répétitions))

Exemple : le codage de la chaîne "AAAMMRRRRXAA" donne la chaîne "A3M2R4X1A2".

Dans ce qui suit, on se propose d'étudier l'algorithme RLE pour compresser des chaînes de caractères.

- On appelle **chaîne d'origine**, toute chaîne de caractère avant sa compression.
- On appelle **chaîne compressée de la chaîne d'origine**, la chaîne de caractère contenant un seul caractère de toute suite de caractères identiques consécutifs de la chaîne d'origine.

Exemple : la chaîne compressée de la chaîne d'origine "FFFFSSBBFFAAA" est : "FSBFA".

Question 1 : Ecrire une fonction d'entête : **def Compresser (S) :** qui permet de construire puis retourner la chaîne compressée (SC) de la chaîne d'origine S.

- Soient S une chaîne d'origine et SC sa chaîne compressée. On appelle liste d'occurrences de SC dans S, une liste d'entiers contenant le nombre des répétitions successives de chaque caractère de SC dans S.

Exemple : Si S="VVVVUCCCCJHHJJJJ" et SC="VUCJHJ", alors la liste d'occurrences de SC dans S est : L= [4, 2, 3, 1, 2, 5]

Question 2 : Ecrire une fonction d'entête : **def Occurrence (S, SC) :** qui permet de remplir la liste L (occurrences de SC dans S)

Question 3 : Ecrire une fonction d'entête : **def Decompresser (SC, L) :** qui permet de décompresser la chaîne SC en chaîne d'origine S.

Exemple : Si SC= "TDNIS" et L= [1, 4, 2, 1, 3] alors l'appel à la fonction **Decompresser(SC,L)** retourne la chaîne S="TDDDDNNISSS"

Question 4 : Ecrire une fonction d'entête : **def Coder (S) :** qui permet de mettre dans la chaîne SC la chaîne codée de S. Exemple : Si S= "MMMMMDDDXRTTT" alors l'appel à la fonction **Coder (S)** retourne la chaîne SC="M5D3X2R1T3".

Question 5 : Ecrire une fonction d'entête : **def Decoder (SC) :** qui permet de mettre dans la chaîne S la chaîne d'origine de SS. Exemple : Si SC="M5D3X2R1T3" alors l'appel à la fonction **Decoder (SC)** retourne la chaîne S= "MMMMMDDDXRTTT".